Machine Learning and Pattern Recognition

L04: Training Models

Prof. Dr. Mohammed Elmogy

Mansoura University
Faculty of Computers and Information
Information Technology Department

melmogy@mans.edu.eg

01. November 2025

Outline

Introduction
Linear Regression
- The Normal Equation
- Computational Complexity

Gradient Descent
- Batch Gradient Descent
- Stochastic Gradient Descent
- Mini-batch Gradient Descent

Outline

Introduction
Linear Regression
Gradient Descent

Good Understanding of How Things Work

- Quickly home in on the appropriate model.
- The right training algorithm to use.
- A good set of hyperparameters for your task.
- Help you debug issues and perform error analysis more efficiently.

Simple Ways to Train Linear Regression

- Using a direct closed-form equation that **directly computes the model parameters** that best fit the model to the training set.
- Using an iterative optimization approach called Gradient Descent (GD) that **gradually tweaks the model parameters** to **minimize the cost function** over the training set.

Outline

Introduction
Linear Regression
The Normal Equation
Computational Complexity
Gradient Descent

Linear Regression Model Prediction

- It is a linear model makes a prediction by simply computing a weighted sum of the input features, plus a constant called the bias (intercept) term.

- $\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + ... + \theta_n x_n$
  - $\hat{y}$ is the predicted value.
  - n is the number of features.
  - x_i is the ith feature value.
  - θ_j is the jth model parameter (**bias + feature weights**).

Linear Regression Model Prediction: Vectorized Form

- $\hat{y} = h_\theta(\mathbf{x}) = \theta.\mathbf{x}$
  - θ is the model's parameter vector.
  - $\mathbf{x}$ is the instance's feature vector.
  - $\theta.\mathbf{x}$ is the dot product of the vectors θ and $\mathbf{x}$.
  - h_θ is the hypothesis function.

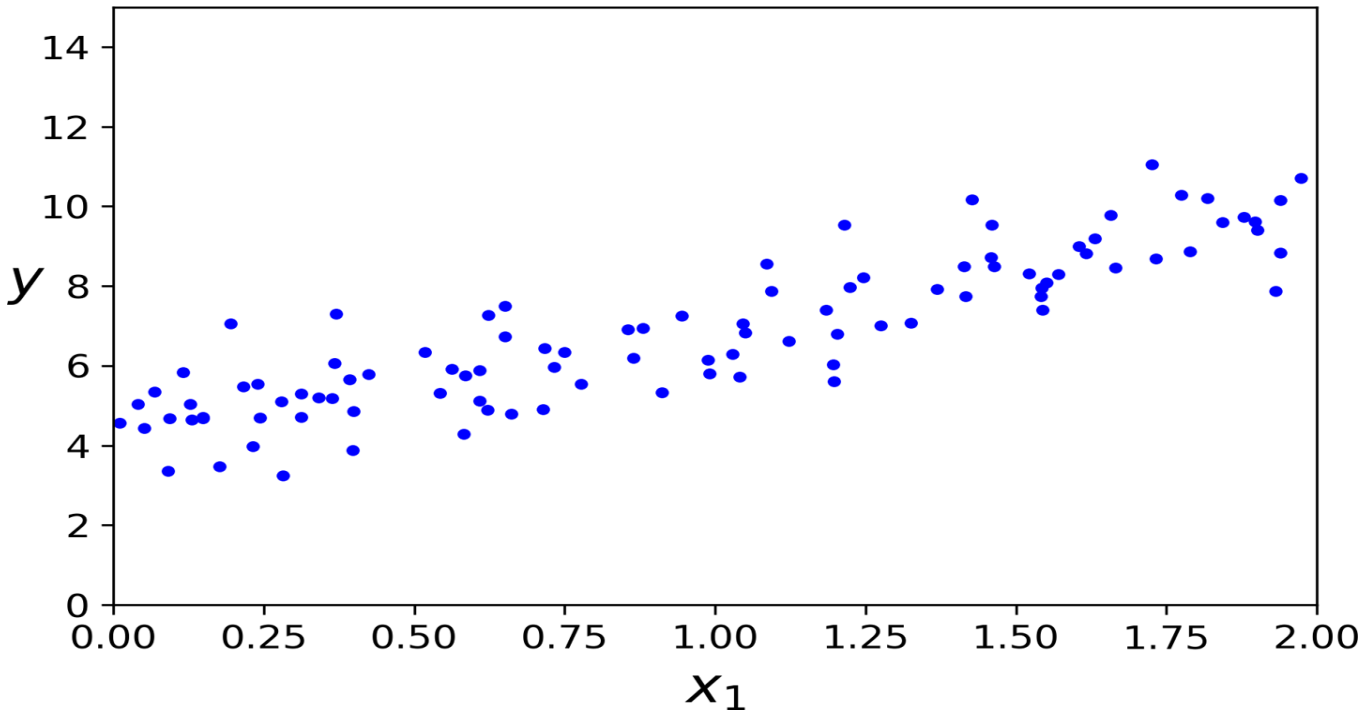
Training a Model

- It means setting its parameters so that the model best fits the training set.
- To train a Linear Regression model, we need to find the value of θ that **minimizes the Root Mean Square Error (RMSE)**.
- In practice, it is simpler to minimize the mean squared error (MSE) than the RMSE, and it leads to the same result.
- $MSE(\mathbf{X}, h_\theta) = MSE(\theta) = \frac{1}{m}\sum_{i=1}^{m}(\theta^T \mathbf{x}^{(i)} - y^{(i)})^2$

The Normal Equation

- To find the value of θ that minimizes the cost function, there is a **closed-form solution**.
- The Normal Equation: $\hat{\theta} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$
  - $\hat{\theta}$ is the value of θ that **minimizes the cost function**.
  - $\mathbf{y}$ is the vector of **target values** containing $y^{(1)}$ to $y^{(m)}$.
- The function that we used to generate the data is $y = 4 + 3x_1$ + Gaussian noise.

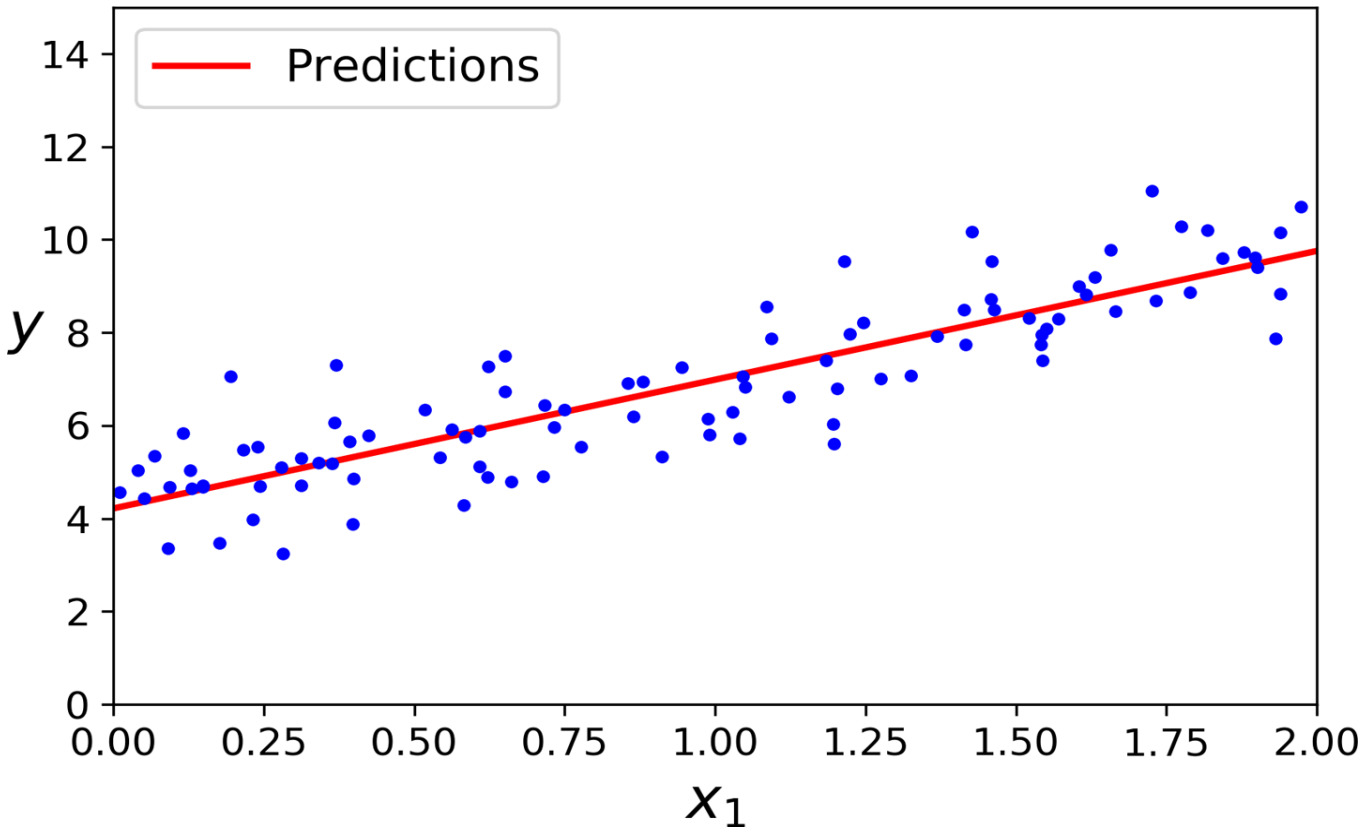
Randomly Generated Linear Dataset




```
import numpy as np

X = 2 * np.random.rand(100, 1)
y = 4 + 3 * X + np.random.randn(100, 1)
```

Linear Regression Model Predictions



Singular Value Decomposition (SVD)

- LinearRegression computes $\hat{\theta} = \mathbf{X}^{+}\mathbf{y}$, where $\mathbf{X}^{+}$ is the pseudoinverse of $\mathbf{X}$ (the Moore-Penrose inverse).
- The pseudoinverse itself is computed using a **standard matrix factorization technique** called SVD.
- SVD can decompose the **training set matrix X** into the **matrix multiplication of three matrices $\mathbf{U\Sigma V^T}$**.

Singular Value Decomposition (SVD) (cont'd)

- The pseudoinverse is computed as $\mathbf{X}^{+} = \mathbf{V}\boldsymbol{\Sigma}^{+}\mathbf{U}^{\mathbf{T}}$.
- This approach is **more efficient than computing the Normal Equation**, plus it **handles edge cases nicely**.
- Indeed, the Normal Equation may not work if the matrix $\mathbf{X}^{\mathbf{T}}\mathbf{X}$ is not invertible (i.e., singular), such as if $m < n$ or if some features are **redundant**, but the pseudoinverse is always defined.

Computational Complexity

- The Normal Equation computes the inverse of $\mathbf{X}^T\mathbf{X}$, which is an $(n+1) \times (n+1)$ matrix (where **n** is the number of features). The computational complexity of inverting such a matrix is typically about $O(n^{2.4})$ to $O(n^3)$, depending on the implementation.
- The SVD approach used by Scikit-Learn's LinearRegression class is about $O(n^2)$.
- Once you have trained your Linear Regression model (using the Normal Equation or any other algorithm), predictions are very fast: the computational complexity is linear with regard to both the number of instances you want to make predictions on and the number of features.

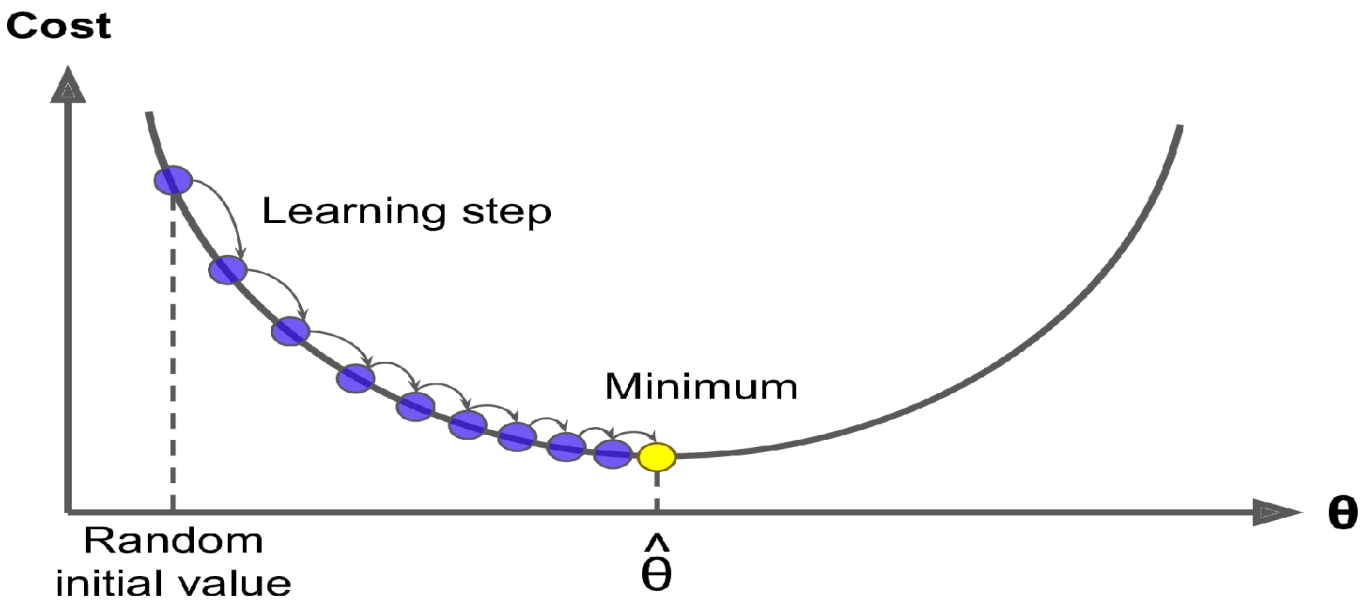
Outline

Introduction
Linear Regression
Gradient Descent
- Batch Gradient Descent
- Stochastic Gradient Descent
- Mini-batch Gradient Descent

Gradient Descent (GD)

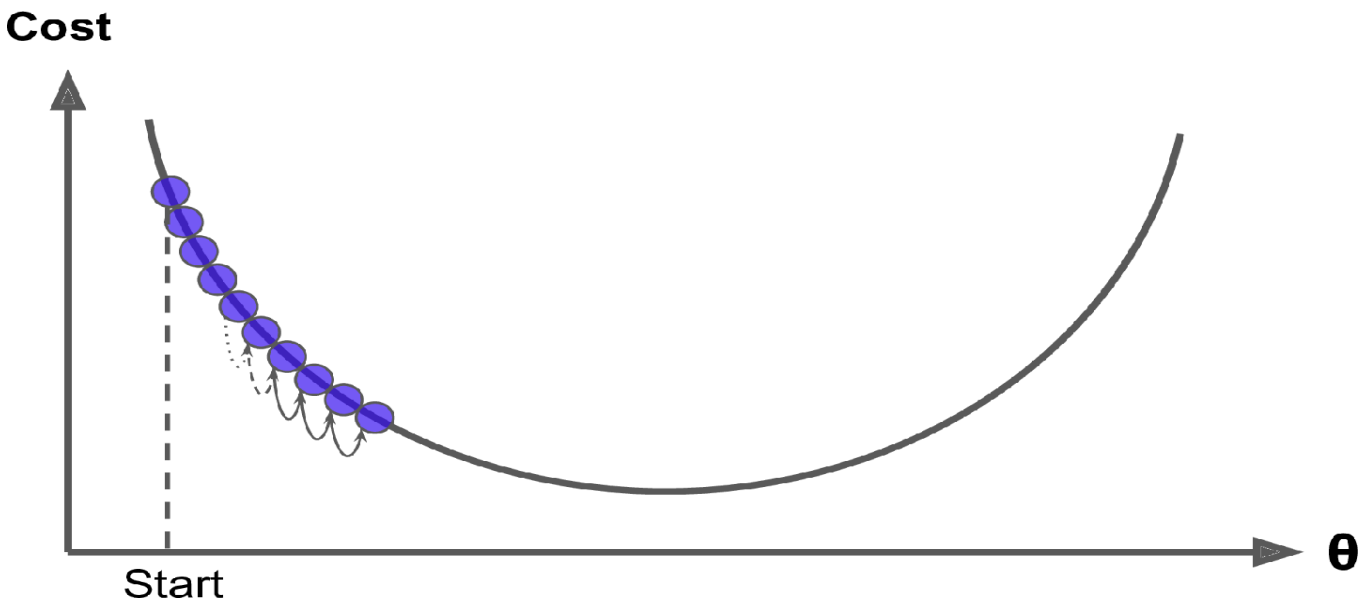
- GD is a generic optimization algorithm capable of **finding optimal solutions** to a wide range of problems.
- The general idea of GD is to tweak parameters iteratively in order to minimize a cost function.
- It measures the local gradient of the error function with regard to the parameter vector θ, and it goes in the direction of descending gradient. Once the gradient is zero, you have reached a minimum.

Gradient Descent (GD) (cont'd)



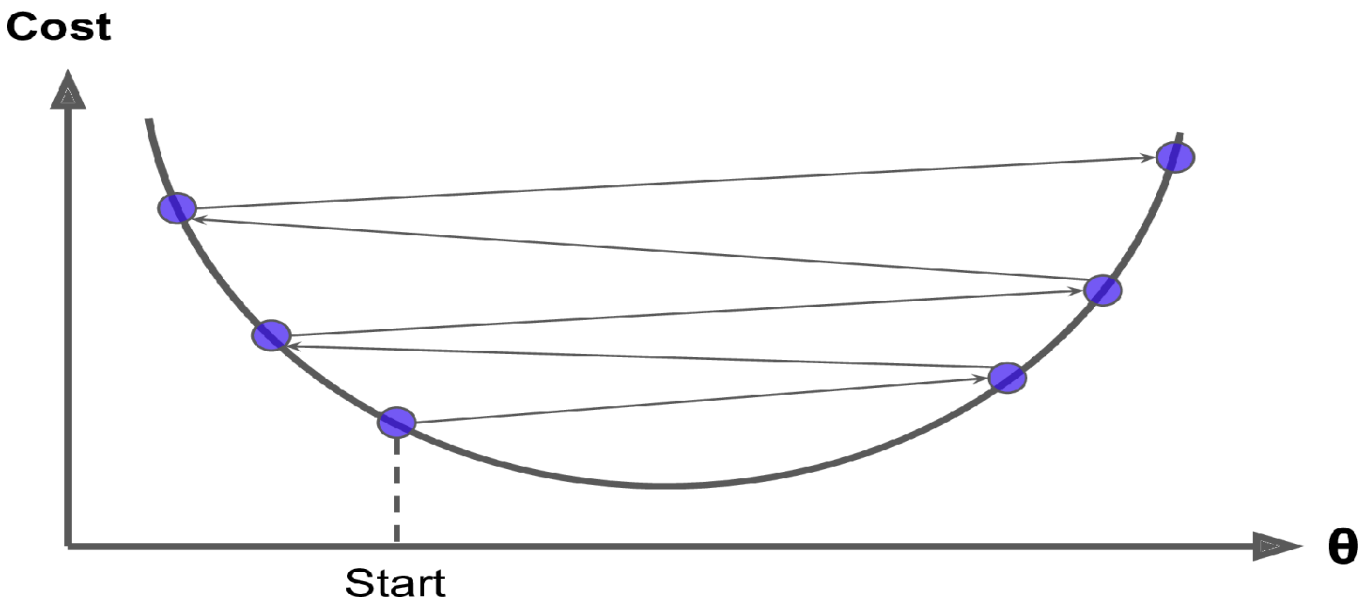
- θ starts with **random values**.
- An important parameter in GD is the size of the steps, determined by the learning rate hyperparameter.

The Learning Rate is too Small



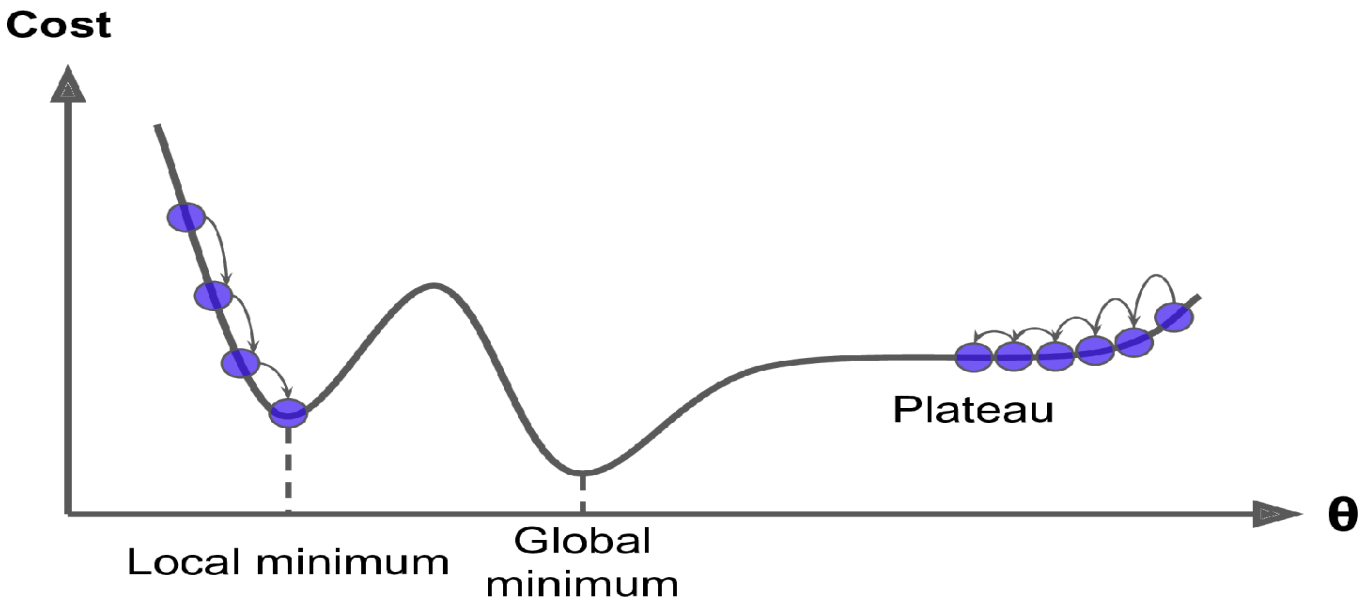
- If the learning rate is too small, then the algorithm will have to go through many iterations to converge, which will take a long time.

The Learning Rate is too Large



- If the learning rate is too high, you might jump across the valley and end up on the other side, possibly even higher up than you were before $\Longrightarrow$ failing to find a good solution.

Main Challenges with GD



- **Left**: It will converge to a local minimum.
- **Right**: It will take a very long time to cross the plateau.

Notes

- The MSE cost function for a Linear Regression model happens to be a convex function.
- Convex function means that if you pick any two points on the curve, the line segment joining them never crosses the curve. It is also a continuous function with a slope that never changes abruptly.

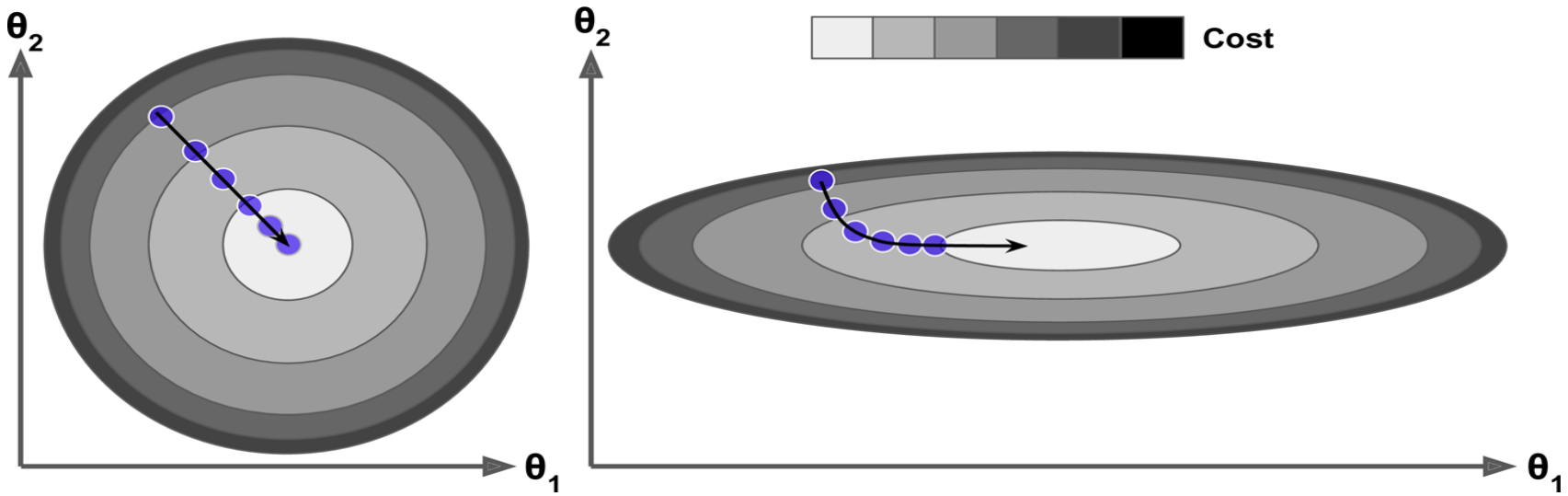
GD is guaranteed to approach arbitrarily close the global minimum (wait long enough + learning rate is not too high).

Notes

- The MSE cost function for a Linear Regression model happens to be a convex function.
- Convex function means that if you pick any two points on the curve, the line segment joining them never crosses the curve. It is also a continuous function with a slope that never changes abruptly.

GD is guaranteed to approach arbitrarily close the global minimum (wait long enough + learning rate is not too high).

GD with and without Feature Scaling



- The cost function has the shape of a bowl, but it can be an elongated bowl if the features have very different scales.
- When using GD, you should ensure that all features have a similar scale.

1. Batch Gradient Descent

- You need to compute the gradient of the cost function with regard to each model parameter $\theta_j \Longrightarrow$ Partial Derivative.
- **Partial derivatives of the cost function**:
  $\frac{\partial}{\partial \theta_j} MSE(\theta) = \frac{2}{m} \sum_{i=1}^{m} (\theta^T \mathbf{x}^{(i)} - y^{(i)}) x_j^{(i)}$
- The gradient vector, noted $\bigtriangledown_\theta MSE(\theta)$, **contains all the partial derivatives of the cost function** (one for each model parameter).

Gradient Vector of the Cost Function

$$\nabla_{\boldsymbol{\theta}} \mathrm{MSE}(\boldsymbol{\theta}) = \begin{pmatrix} \frac{\partial}{\partial \theta_0} \mathrm{MSE}(\boldsymbol{\theta}) \\ \frac{\partial}{\partial \theta_1} \mathrm{MSE}(\boldsymbol{\theta}) \\ \vdots \\ \frac{\partial}{\partial \theta_n} \mathrm{MSE}(\boldsymbol{\theta}) \end{pmatrix} = \frac{2}{m} \mathbf{X}^\intercal (\mathbf{X}\boldsymbol{\theta} - \mathbf{y})$$

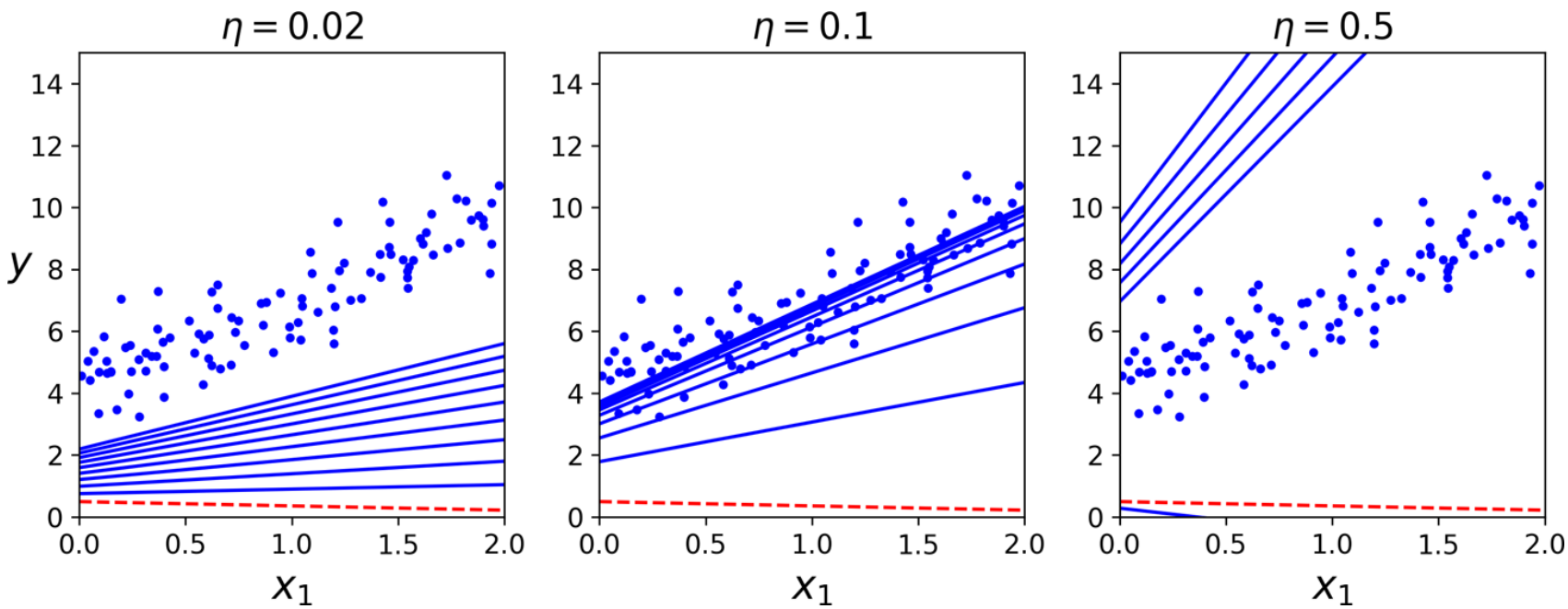
- Batch Gradient Descent uses the **whole batch of training data at every step**.
- As a result, it is terribly slow on very large training sets.

GD Step

- Training a Linear Regression model when there are hundreds of thousands of features is **much faster** using Gradient Descent than using the Normal Equation or SVD decomposition.

- Once you have the gradient vector, which points uphill, just go in the opposite direction to go downhill:
$\theta^{(nextStep)} = \theta - \eta \bigtriangledown_\theta MSE(\theta)$

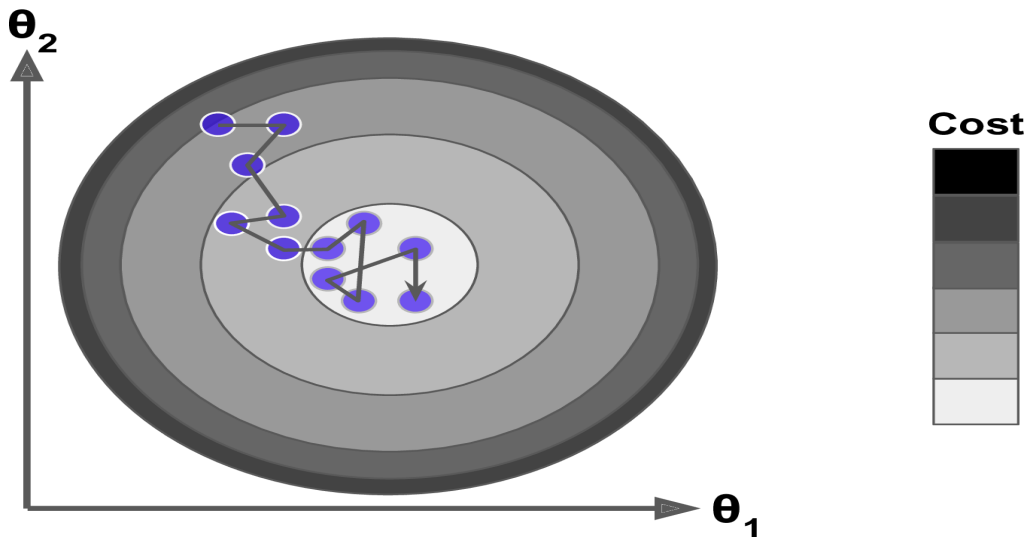
GDt with Various Learning Rates



2. Stochastic Gradient Descent (SGD)

- SGD picks a random instance in the training set at every step and computes the gradients **based only on that single instance**.
- This makes the algorithm much faster because it has **very little data to manipulate** at every iteration.
- It also makes it possible to train on huge training sets, since only one instance needs to be in memory at each iteration.

2. Stochastic Gradient Descent (SGD) (cont'd)



- Over time, it will end up very close to the minimum, but once it gets there **it will continue to bounce around, never settling down**.
- So, once the algorithm stops, the final parameter values are good, but not optimal.

Simulated Annealing

- SGD has a better chance of finding the global minimum than Batch Gradient Descent does.
- Randomness is **good to escape from local optima**, but bad because it means that the algorithm can never settle at the minimum. One solution to this dilemma is to gradually reduce the learning rate $\Longrightarrow$ Simulated Annealing.
- Simulated Annealing inspired from the process in **metallurgy of annealing**, where molten metal is slowly cooled down.
- The function that determines the learning rate at each iteration is called the learning schedule.

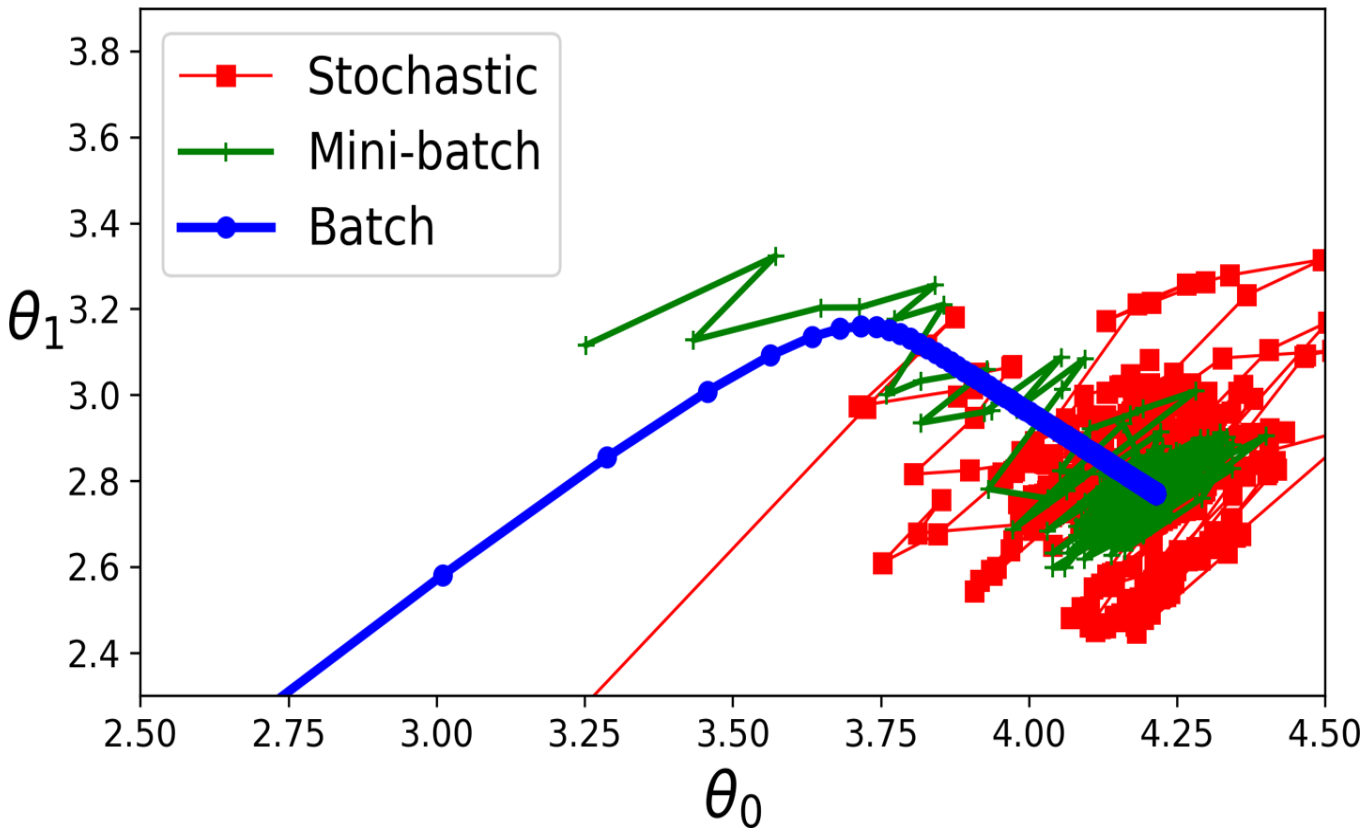
Notes

- Note that since instances are picked randomly, some instances may be picked several times per epoch, while others may not be picked at all.
- When using SGD, the training instances must be independent and identically distributed (IID) to ensure that the parameters get pulled toward the global optimum, on average.
- A simple way to ensure this is to shuffle the instances during training.

3. Mini-batch Gradient Descent

- Mini-batch GD computes the **gradients on small random sets** of instances called mini-batches.
- Mini-batch GD will end up walking around a bit closer to the minimum than Stochastic GD, but it may be harder for it to escape from local minima.

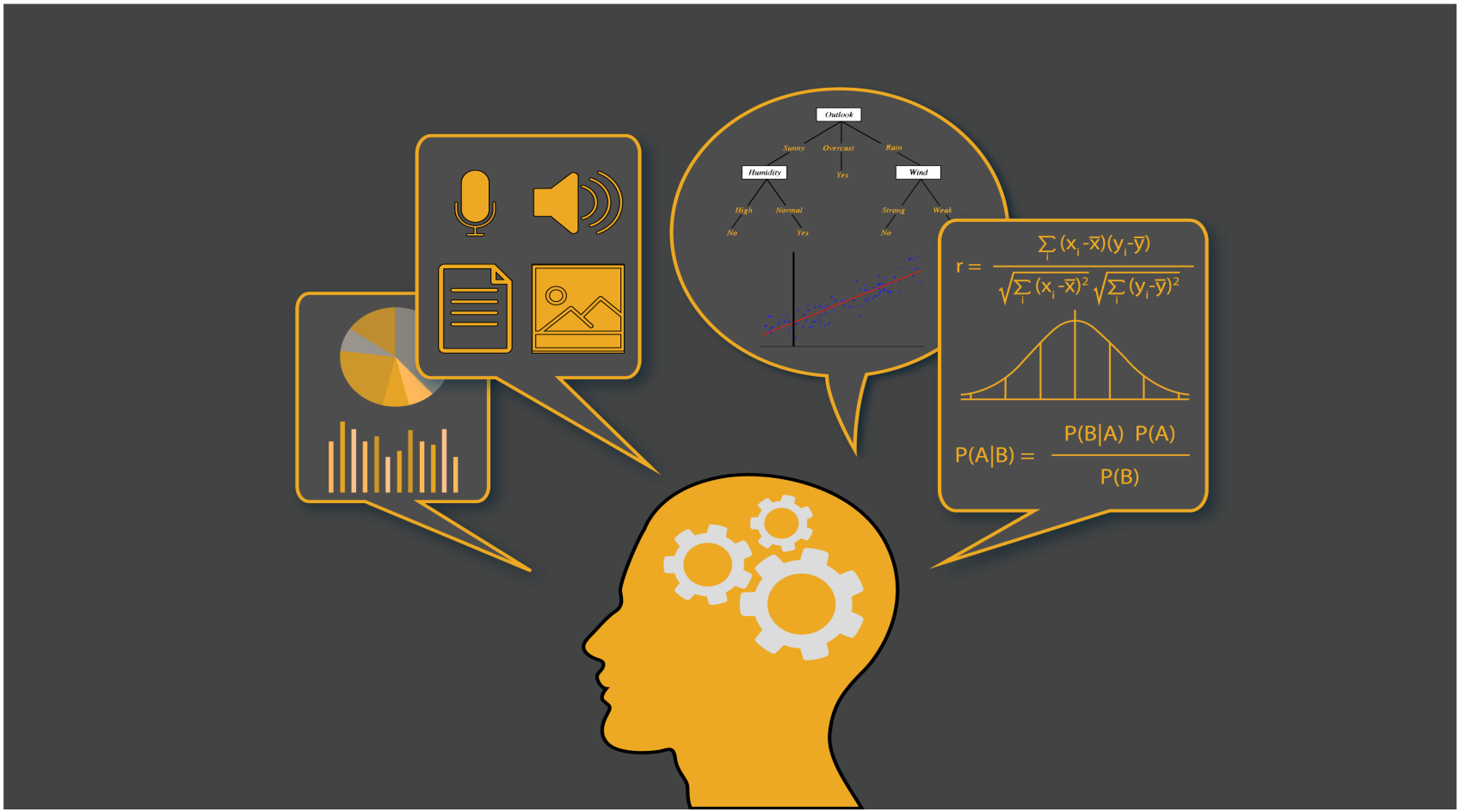
Gradient Descent Paths in Parameter Space



Comparison of Algorithms for Linear Regression

Algorithm	Large m	Out-of-core support	Large n	Hyperparams	Scaling required	Scikit-Learn
Normal Equation	Fast	No	Slow	0	No	N/A
SVD	Fast	No	Slow	0	No	`LinearRegression`
Batch GD	Slow	No	Fast	2	Yes	`SGDRegressor`
Stochastic GD	Fast	Yes	Fast	≥2	Yes	`SGDRegressor`
Mini-batch GD	Fast	Yes	Fast	≥2	Yes	`SGDRegressor`

- There is almost no difference after training: all these algorithms **end up with very similar models and make predictions in exactly the same way**.


Outlook
Sunny
Overcast
Rain
Humidity
Yes
Wind
High
Normal
Strong
Weak
No
Yes
No
Yes
$r = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_i (x_i - \bar{x})^2} \sqrt{\sum_i (y_i - \bar{y})^2}}$
$P(A|B) = \frac{P(B|A)\ P(A)}{P(B)}$